

Week 6: ServerWatch APM & AI Diagnostics Portal

Capstone Project Architecture, Implementation & DevOps Guide

Week 6: The Capstone Project - ServerWatch APM & AI

Diagnostics Portal

The Complete Architecture, Implementation, and DevOps

Troubleshooting Guide

Welcome to the Week 6 Capstone Project Guide. This document consolidates the entire ServerWatch Application Performance Monitoring (APM) and SLA Audit Portal. It outlines the core architectural components, explains the implementation files, details the AWS services utilized, and prepares you to answer any conceptual or operational questions during your DevOps interview with the Operations Head.

Table of Contents

1. Module 1: The Business Problem & APM Requirements
2. Module 2: Serverless Decoupled Telemetry Pipeline (API Gateway → SQS → Lambda)
3. Module 3: Database Optimization with DynamoDB Single-Table Design
4. Module 4: Code Implementation Walkthrough (Ingestor, Processor, Auditor)
5. Module 5: Security & Role-Based Access Control (Amazon Cognito JWT Auth)
6. Module 6: Frontend Architecture & Layout Margin Polishing
7. Module 7: Generative AI Diagnostics (Google Gemini 2.5 Flash Integration)
8. Module 8: Infrastructure as Code (Terraform IaC) Resource Declarations
9. Module 9: Cost-Optimization & Resiliency Measures
10. Module 10: Mock Interview Preparation (Ops-Head Target Q&A)

Module 1: The Business Problem & APM Requirements

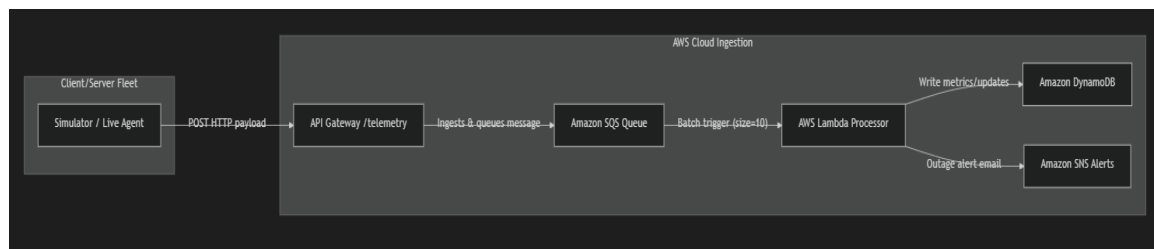
In modern enterprise architectures, applications are split into multiple microservices running in virtual containers or serverless runtimes. Finding why an

application is slow, diagnosing which component crashed, and calculating if the services are meeting their Service Level Agreements (SLAs) is difficult.

ServerWatch solves these problems by providing:

- Real-time Telemetry Ingestion: Stream CPU, Memory, and Latency statistics directly from running service nodes.
- Dynamic Multi-Metric Threshold Alerting: Check incoming metrics against server-specific limits and send instant alerts when a resource is overloaded.
- Daily SLA Compliance Logging: Aggregate performance logs daily to calculate uptime vs outage trends.
- AI-Powered Diagnostics: Enable engineers to query log indexes in natural language using large language models (LLMs) to troubleshoot root causes.

Module 2: Serverless Decoupled Telemetry Pipeline



The data ingestion pipeline utilizes a decoupled publisher-subscriber pattern:

- The Ingestion Proxy (`lambda/ingestion.py`): A lightweight handler sits behind API Gateway. When it receives a request, it runs simple syntax checks and transfers the message directly to Amazon SQS. It does not perform database writes or calculations, returning a quick `202 Accepted` response.
- The Shock Absorber (Amazon SQS): Direct database writes during network surges lead to service throttling and data loss. SQS acts as a buffer. Under spike loads, SQS queues messages safely in-flight, shielding our processing layer.
- The Worker (`lambda/processor.py`): AWS Lambda retrieves SQS messages in batches (`size=10`), processes the thresholds, updates the database, and publishes notifications using Amazon SNS if anomalies occur.

Module 3: Database Optimization with DynamoDB Single-Table Design

A standard database structure utilizes multiple tables (e.g. ``raw_logs``, ``daily_summaries``). However, querying multiple tables incurs separate index lookups and network roundtrips. ServerWatch implements DynamoDB Single-Table Design, keeping all telemetry logs and daily reports in a single physical table (`smartwatt-telemetry-dev``).

3.1 Partition Key (PK) & Sort Key (SK) Index Design

Row Type	PK (Partition Key)	SK (Sort Key)	Attributes
Telemetry Log	SERVER# {device_id}	LOG# {timestamp }	cpu_utilization, memory_utilization, latency, submitted_by
Daily SLA Summary	REPORT#DAILY	DATE# {report_date}	total_active_devices, gateway_spikes_count, cpu_overloads_count, memory_crashes_count, active_devices (set), anomaly_devices (set)

- Efficient Fetching: When looking up logs for ``api-gateway-prod``, the query filters on ``PK="SERVER#api-gateway-prod"``` and gets all sorted logs automatically.
- Daily Summary Query: When loading the dashboard charts, the client queries ``PK="REPORT#DAILY"```, fetching the historical audits list in one query.

Module 4: Code Implementation Walkthrough

4.1 Ingestion Lambda (``lambda/ingestion.py``)

This script acts as the SQS producer proxy.

```
import json

import os

import boto3

sqs = boto3.client('sqs')
```

```
QUEUE_URL = os.environ.get('QUEUE_URL')

def lambda_handler(event, context):

    body_str = event.get('body')

    if not body_str:

        return {"statusCode": 400, "body": json.dumps({"error":
        "Missing payload body"})}

    try:

        data = json.loads(body_str)

    except Exception:

        return {"statusCode": 400, "body": json.dumps({"error":
        "Invalid JSON format"})}

    # Strict validation check

    required = ['device_id', 'latency', 'cpu_utilization',
    'memory_utilization']

    for field in required:

        if field not in data:

            return {"statusCode": 400, "body": json.dumps({"error":
            f"Missing field: {field}"})}

    payload = {

        "device_id": data['device_id'],

        "latency": float(data['latency']),

        "cpu_utilization": float(data['cpu_utilization']),
```

```
"memory_utilization": float(data['memory_utilization']),
"timestamp": data.get('timestamp') or context.aws_request_id,
"submitted_by": data.get('submitted_by') or "System
Simulator"
}

sqs.send_message(
    QueueUrl=QUEUE_URL,
    MessageBody=json.dumps(payload)
)

return {
    "statusCode": 202,
    "headers": {
        "Access-Control-Allow-Origin": "*",
        "Access-Control-Allow-Headers": "Content-Type",
        "Access-Control-Allow-Methods": "POST,OPTIONS"
    },
    "body": json.dumps({"message": "Telemetry accepted and
queued"})
}
```

4.2 Processing Lambda (`lambda/processor.py`)

This script processes batches of messages, evaluates dynamic thresholds, writes logs, and logs outages.

```
import json

import os

import decimal

import boto3

from datetime import datetime

dynamodb = boto3.resource('dynamodb')

sns = boto3.client('sns')

TABLE_NAME = os.environ.get('DYNAMODB_TABLE_NAME')

SNS_TOPIC_ARN = os.environ.get('SNS_TOPIC_ARN')

# Server-specific dynamic limits

DEVICE_THRESHOLDS = {

    "api-gateway-prod": {"latency": 250.0, "cpu": 80.0, "memory":
85.0},

    "auth-service-pod": {"latency": 150.0, "cpu": 90.0, "memory":
90.0},

    "pdf-generator-worker": {"latency": 500.0, "cpu": 85.0,
"memory": 95.0},

    "default": {"latency": 250.0, "cpu": 90.0, "memory": 95.0}

}

def lambda_handler(event, context):
```

```
table = dynamodb.Table(TABLE_NAME)

for record in event.get('Records', []):

    body = json.loads(record['body'])

    device_id = body['device_id']

    latency = float(body['latency'])

    cpu = float(body['cpu_utilization'])

    mem = float(body['memory_utilization'])

    timestamp = body['timestamp']

    submitter = body['submitted_by']

    # Write individual server telemetry log

    table.put_item(

        Item={

            'PK': f"SERVER#{device_id}",

            'SK': f"LOG#{timestamp}",

            'device_id': device_id,

            'latency': decimal.Decimal(str(latency)),

            'cpu_utilization': decimal.Decimal(str(cpu)),

            'memory_utilization': decimal.Decimal(str(mem)),

            'timestamp': timestamp,

            'submitted_by': submitter

        }
```



```

)

# Load custom thresholds

limits = DEVICE_THRESHOLDS.get(device_id,
DEVICE_THRESHOLDS['default'])

is_anomaly = False

reasons = []

if latency > limits['latency']:

is_anomaly = True

reasons.append(f"Latency spike: {latency}ms (Limit:
{limits['latency']}ms)")

if cpu > limits['cpu']:

is_anomaly = True

reasons.append(f"CPU overload: {cpu}% (Limit:
{limits['cpu']}%)")

if mem > limits['memory']:

is_anomaly = True

reasons.append(f"Memory overload: {mem}% (Limit:
{limits['memory']}%)")

# Send instant alarm if anomaly found

if is_anomaly and SNS_TOPIC_ARN:

sns.publish(

TopicArn=SNS_TOPIC_ARN,

```

```

Subject=f"ALERT: SLA Breach on {device_id}",

Message=f"An incident occurred on {device_id} at
{timestamp}.\n\nDetails:\n" + "\n".join(reasons)
)

# Atomic counter updates for daily SLA audits

date_str = timestamp.split('T')[0]

update_expr = "ADD total_active_devices :val_one,
active_devices :dev_set"

expr_vals = {

":val_one": decimal.Decimal('1'),

":dev_set": {device_id}

}

if is_anomaly:

update_expr += ", anomaly_devices :anom_set"

expr_vals[":anom_set"] = {device_id}

# Map specific incident counters

if latency > limits['latency']:

update_expr += ", gateway_spikes_count :val_one"

if cpu > limits['cpu']:

update_expr += ", cpu_overloads_count :val_one"

if mem > limits['memory']:

update_expr += ", memory_crashes_count :val_one"

```

```

table.update_item(

Key={'PK': 'REPORT#DAILY', 'SK': f"DATE#{date_str}"},

UpdateExpression=update_expr,

ExpressionAttributeValues=expr_vals

)

return {"statusCode": 200, "body": "Batch processed
successfully"}

```

Module 5: Security & Role-Based Access Control

AWS Cognito secures the platform by controlling user accounts and verifying resource queries.

- User Identity verification: Users sign in via Cognito User Pools.
- Access Control Tokens: Cognito returns a JWT (JSON Web Token) ID Token. Inside this token are claims (such as `email` and `custom:role`).
- API Authorizer Check: When the dashboard calls API Gateway to load audits, the gateway intercepts the request and verifies the signature of the JWT token.
- Backend Role Enforcement: The Auditor Lambda extracts the claims:

```

claims = event.get('requestContext', {}).get('authorizer',
{}).get('claims', {})

user_role = claims.get('custom:role')

user_email = claims.get('email')

if user_role != 'Admin' and user_email !=
'vaibhavsp16@gmail.com':

    return {"statusCode": 403, "body": "Access Denied"}

```

If validation checks fail, it blocks the query immediately.

Module 6: Frontend Architecture & Layout Margin Polishing

The frontend is a single-page HTML web application stored in Amazon S3 and distributed through Amazon CloudFront.

6.1 UI Margin Polish

To prevent elements from sticking directly to the screen boundaries on wide screens, a centered flex wrapper container is used to manage page alignments:

```
<div id="mainContent" style="flex: 1; padding: 25px 0;
display: flex; flex-direction: column; width: 100%;">

<div style="max-width: 1200px; width: 100%; margin: 0 auto;
padding: 0 25px; box-sizing: border-box; display: flex; flex-
direction: column; flex: 1;">

<!-- Header, Feed, Charts, and Chat tabs sit here -->

</div>

</div>
```

- Centering: `margin: 0 auto` automatically centers the layout on large displays.
- Margins: `padding: 0 25px` ensures a minimum spacing from the screen boundary on compact viewports.
- Nesting Integrity: All tabs (Dashboard, Visual Stats, AI DevOps Bot) sit inside this wrapper, guaranteeing consistent alignment across the entire portal.

Module 7: Generative AI Diagnostics (Gemini API Integration)

The AI DevOps Bot tab integrates the Google Gemini 2.5 Flash API directly within the client interface to assist engineers in diagnosing incidents.

```
[Raw Log Audits in DynamoDB]
|
▼ (Fetch via Lambda API)
[JSON Logs Stringified] → [Gemini 2.5 Flash Model Prompt] → [Clean Plain Text Response]
```

- Context Loading: The application fetches the active database logs.
- Instruction Prompting: The app sends these logs to Gemini with instructions:
 1. Analyze the logs for resource warnings, memory overloads, or latency spikes.
 2. Output recommendations in plain text only.
 3. Do NOT use markdown bold stars or headers. Use solid round dots (•) for lists.
- Response Polishing: A frontend regex function (cleanAiText()) strips any remaining markdown formatting, outputting clean, readable diagnostics.

Module 8: Infrastructure as Code (Terraform)

The architecture is fully managed via Terraform.

8.1 Key Resource blocks in `main.tf`

- Amazon SQS Buffer Queue:

```
resource "aws_sqs_queue" "telemetry_queue" {
    name                                = "smartwatt-queue-dev"
    visibility_timeout_seconds = 30
    redrive_policy = jsonencode({
        deadLetterTargetArn = aws_sqs_queue.dlq.arn
        maxReceiveCount      = 3
    })
}
```

- Cognito User Pool Client:

```
resource "aws_cognito_user_pool_client" "client" {  
    name          = "smartwatt-client-dev"  
    user_pool_id  = aws_cognito_user_pool.user_pool.id  
    generate_secret = false  
    read_attributes = ["email", "custom:role"]  
    write_attributes = ["email", "custom:role"]  
}
```

- DynamoDB Single-Table:

```
resource "aws_dynamodb_table" "telemetry_table" {  
    name          = "smartwatt-telemetry-dev"  
    billing_mode  = "PAY_PER_REQUEST"  
    hash_key     = "PK"  
    range_key    = "SK"  
    attribute { name = "PK"; type = "S" }  
    attribute { name = "SK"; type = "S" }  
}
```

Module 9: Cost-Optimization & Resiliency Measures

A critical requirement of professional systems is cost management.

- On-Demand Capacity: Setting DynamoDB capacity to `PAY\_PER\_REQUEST` ensures you pay only for actual read/write invocations, incurring \$0.00 cost when idle.
- SQS Message Batching: Processing SQS payloads in batches of 10 reduces the total number of Lambda invocations by 90%, keeping computing fees within the AWS free tier.
- Dead Letter Queue (DLQ): If a payload is corrupt, the system stops trying to process it after 3 failures and routes it to a DLQ. This prevents infinite processing loops that drive up Lambda execution costs.

Module 10: Mock Interview Preparation (Ops-Head Target Q&A)

Q1: "How does this system handle a sudden 10x spike in server metrics traffic?"

Response: "Because we decoupled ingestion from processing using Amazon SQS, traffic spikes pose no risk to database stability. API Gateway queues incoming payloads directly into SQS. The queue buffers the traffic in flight, and our Lambda processor pulls messages in controlled batches (size=10) without throttling or dropping data."

Q2: "How is database access restricted so only administrators can view the audit reports?"

Response: "We implement token authentication using Amazon Cognito. When a user requests report logs, the frontend attaches their Cognito JWT token in the API request header. API Gateway validates the token signature, and the Auditor Lambda checks the JWT claims. If the user's `custom:role` is not `Admin`, the Lambda rejects the query with an HTTP `403 Forbidden` response."

Q3: "What happens if the processor Lambda encounters a malformed payload? How do we prevent infinite loops?"

Response: "We implemented a SQS Dead Letter Queue (DLQ). If a payload fails to process due to corruption, the redrive policy retries it up to 3 times. If it continues to fail, SQS automatically moves it to the DLQ, halting the loop and triggering alerts so DevOps engineers can inspect it without driving up execution costs."

Q4: "How does the single-table design optimize query performance?"

Response: "Traditional multi-table databases require complex joins and multiple requests to correlate raw logs with daily summaries. With our single-table design, raw logs and summaries reside in the same physical table, separated by key partition patterns (`SERVER#{id}` vs `REPORT#DAILY`). We can retrieve complete report listings or detailed metric timelines with a single quick index query."